Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Algorithmes de recherche et complexité algorithmique dans les structures linéaires

Synthèse exécutive : Ce cours explore les deux méthodes fondamentales de recherche d'une donnée dans un tableau : la recherche séquentielle (ou linéaire) et la recherche dichotomique. L'enjeu clé repose sur la compréhension du lien direct entre la structuration des données (tableau trié ou non trié) et l'efficacité temporelle du traitement. Alors que la recherche séquentielle parcourt l'ensemble des éléments un à un avec une complexité linéaire O(n), la recherche dichotomique divise par deux l'espace de recherche à chaque étape grâce au principe 'diviser pour régner', atteignant une complexité logarithmique O(log n). Ce cours formalise l'exécution pas à pas de ces deux algorithmes, l'évolution des pointeurs d'indices, ainsi que les règles d'arbitrage économique et algorithmique pour choisir la méthode adaptée.

La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est l'approche la plus intuitive pour trouver un élément dans un tableau. L'algorithme initialise un pointeur d'indice à 0 et examine successivement chaque cellule. À chaque itération, il compare le contenu de la case courante avec la valeur cible recherchée. Si une égalité est détectée, le parcours s'arrête et l'indice est immédiatement renvoyé (succès). Si la fin du tableau est atteinte sans trouver la valeur, l'algorithme renvoie un marqueur d'absence (comme -1 ou Faux). Son principal avantage réside dans son absence totale de précondition : elle s'applique à n'importe quelle structure séquentielle non ordonnée. En revanche, pour un tableau de taille n, le pire des cas exige n comparaisons, rendant cette méthode inefficace pour de très grands volumes de données.

- Parcours exhaustif du tableau indice par indice, du début (0) jusqu'à la fin (n-1).
- Absence totale de précondition : fonctionne sur tableau trié ou non trié.
- Complexité temporelle linéaire O(n) dans le pire des cas (élément en dernière position ou absent).
- Arrêt anticipé dès la première occurrence trouvée (succès).

Cas concret enseignant : *Le professeur a utilisé la métaphore de la recherche d'une clé tombée dans un tas de feuilles mortes. Comme les feuilles n'ont aucun ordre de rangement, il n'existe aucune astuce possible : la seule garantie de trouver la clé est de soulever chaque feuille une par me, de la première à la dernière.*

La Recherche Dichotomique (Principe du Diviser pour Régner)

La recherche dichotomique exploite l'ordre préexistant d'un tableau trié pour éliminer massivement des éléments candidats sans même les examiner. L'algorithme délimite la zone de recherche avec deux pointeurs, 'debut' (initialisé à 0) et 'fin' (initialisé à n-1). Il calcule l'indice central par division entière : milieu = (debut + fin) // 2. Il compare ensuite T[milieu] avec la cible : 1) Si T[milieu] == cible, la recherche aboutit immédiatement. 2) Si la cible est strictement inférieure à T[milieu], elle ne peut se trouver que dans la première moitié : le pointeur 'fin' devient 'milieu - 1'. 3) Si la cible est supérieure, elle se trouve dans la seconde moitié : 'debut' devient 'milieu + 1'. L'opération se répète tant que debut <= fin. Grâce à cette réduction exponentielle, là où la recherche séquentielle demanderait 1 000 000 de comparaisons au pire des cas, la recherche dichotomique en demande au maximum 20.

- Précondition stricte et absolue : le tableau doit être préalablement trié.
- Paradigme Diviser pour Régner : réduction de 50 % de la zone de recherche à chaque itération.
- Manipulation de 3 pointeurs d'indices : debut, fin, et milieu = (debut + fin) // 2.
- Complexité temporelle logarithmique O(log2 n) au pire des cas.

Cas concret enseignant : *Exemple du jeu 'Devine un nombre entre 1 et 100'. Proposer 1, 2, 3... est une stratégie séquentielle médiocre. Proposer 50 permet d'éliminer d'un coup la moitié des possibilités selon que la réponse est 'c'est plus grand' ou 'c'est plus petit'. En poursuivant cette logique, on trouve toujours la réponse en 7 coups maximum (2^7 = 128 > 100).*

Arbitrage et Analyse Comparative des Performances

Bien que la recherche dichotomique soit immensément plus performante que la recherche séquentielle, son utilisation exige un tableau trié. Or, trier un tableau non ordonné coûte au mieux O(n log n) opérations. Si l'application ne doit effectuer qu'une seule recherche ponctuelle sur un tableau non trié, appliquer directement une recherche séquentielle O(n) est algorithmiquement plus avantageux que d'exécuter un tri suivi d'une recherche dichotomique O(n log n + log n). Inversement, si la structure de données est stable et doit être interrogée de nombreuses fois, trier le tableau une fois pour toutes constitue un investissement rentable.

- Coût du tri préalable : trier un tableau non ordonné nécessite une complexité O(n log n).
- Recherche unique : sur un tableau non trié, la recherche séquentielle O(n) est plus rapide que Trieur + Dichotomie O(n log n).
- Recherches multiples : le tri initial en O(n log n) est largement amorti si le tableau subit de très nombreuses requêtes de recherche dichotomique en O(log n).

Cas concret enseignant : *L'analogie du dictionnaire imprimé : le travail colossal de mise en ordre alphabétique réalisé lors de la fabrication de l'ouvrage (le tri initial) demande un effort important, mais il permet ensuite à des millions d'utilisateurs d'effectuer des recherches quasi instantanées pendant des années.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique puisqu'elle est beaucoup plus rapide que la recherche séquentielle ?
Réponse du professeur : Parce que la recherche dichotomique nécessite obligatoirement que le tableau soit trié. Si le tableau n'est pas trié, le coût pour le trier (O(n log n)) est plus élevé que le coût d'une simple recherche séquentielle (O(n)). On ne choisit la dichotomie que si le tableau est déjà trié ou si on prévoit de faire de très nombreuses recherches dans le même tableau.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise la division entière (notée // en Python). Par exemple, avec 10 éléments (indices 0 à 9), (0 + 9) // 2 donne 4. La division entière tronque la partie décimale, ce qui choisit automatiquement l'élément central à gauche. L'algorithme fonctionne de façon rigoureusement identique, peu importe que la taille soit paire ou impaire.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] comportant n = 10 éléments (indices 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour localiser la valeur 42.

Résolution pas à pas :
Étape 0 (Initialisation) : debut = 0, fin = 9. Étape 1 : - Calcul du milieu : milieu = (0 + 9) // 2 = 4. - Valeur examinée : T[4] = 19. - Comparaison : 42 > 19. La cible est strictement supérieure, la valeur recherchée se trouve donc dans la sous-partie droite. - Mise à jour du pointeur debut : debut = milieu + 1 = 5. Étape 2 : - Zone de recherche : debut = 5, fin = 9. - Calcul du milieu : milieu = (5 + 9) // 2 = 7. - Valeur examinée : T[7] = 42. - Comparaison : T[7] == 42. La valeur cible est localisée !

Résultat : La valeur 42 a été trouvée à l'indice 7 en exactement 2 comparaisons, alors qu'une recherche séquentielle aurait nécessité 8 comparaisons (de T[0]=3 jusqu'à T[7]=42).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition impérative et indispensable pour pouvoir exécuter une recherche dichotomique sur un tableau ?

R : Le tableau doit être strictement trié au préalable.

Q : Quelle est la complexité temporelle au pire des cas d'une recherche séquentielle sur un tableau de taille n ?

R : Sa complexité est linéaire, notée O(n).

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser un élément dans un tableau trié de 1024 éléments ?

R : 10 comparaisons au maximum, car 2^10 = 1024 (log2(1024) = 10).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765